Quantitative Finance Pricing Engines

Neel Dev Sen

July 2026

Contents

Introduction

This is a project where I develop derivative pricers using **C++** and occasionally **Python**. My goal is to do at least one a day and you can access all of my source files and header files here: **GitHub Repo**

Part I
European Options

1 Closed-Form Solutions

1.1 Black-Scholes Equation

The first option that I am going to be pricing is the European option using the closed-form Black-Scholes equation.

In **GitHub** the full **C++** script is on this link: **source file**
The **header file** (first listing) is on this link: **header file**

1.1.1 Geometric Brownian Motion

The Black-Scholes equation takes the assumption of **Geometric Brownian Motion**

$$dS_t = rS_t dt + \sigma S_t dW_t \tag{1.1}$$

[1]
where S_t is the price of the asset at a given time t, dS_t is the infinitesimal change in the asset's price, r is the risk-free rate of interest, dt is the infinitesimal change in time, σ is the implied volatility and dW_t is the infinitesimal change in the **Wiener function**. The **Wiener function** is defined with the property:

$$W_T - W_t \sim \mathcal{N}(0, T-t) \tag{1.2}$$

where $\mathcal{N}(\mu, \sigma^{\in})$ is the **normal distribution** with mean μ and variance σ^2.[2]

1.1.2 Derivation of the Black-Scholes PDE

Ito's Lemma states that:

$$df(t, S_t) = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}dS_t + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}(dS_t)^2 \tag{1.3}$$

Substituting Equation 1.1 for dS_t we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(rS_t dt + \sigma S_t dW_t\right)^2 \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(r^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 (dt)(dW_t) + \sigma^2 S_t^2 (dW_t)^2\right) \end{aligned} \tag{1.4}$$

The **Ito's multiplication rules** state that:

$$(dt)^2 = 0 \tag{1.5}$$
$$(dt)(dW_t) = 0 \tag{1.6}$$
$$(dW_t)^2 = dt \tag{1.7}$$

Substituting Equations 1.5, 1.6, and 1.7 into Equation 1.4 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(0 + 0 + \sigma^2 S_t^2 (dt)\right) \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\sigma^2 S_t^2 \\ &= (\frac{\partial f}{\partial t} + rS_t\frac{\partial f}{\partial S_t} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 f}{\partial S_t^2})dt + \sigma S_t\frac{\partial f}{\partial S_t}dW_t \end{aligned} \tag{1.8}$$

Define

$$V(t, S) \equiv f(t, S_t) \tag{1.9}$$

Using the result from Equation 1.8

$$dV(t, S) = (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2})dt + \sigma S\frac{\partial V}{\partial S}dW_t \tag{1.10}$$

Now we will define a delta-hedged portfolio which means a portfolio with an option V as well as shorting Δ shares with value S.
Δ is an **option Greek** which is the partial derivative of the value of an option with respect to the underlying asset price $\Delta = \frac{\partial V}{\partial S}$
The value of the portfolio Π is shown through the equation below

$$\Pi = V - \Delta S \tag{1.11}$$

For an infinitesimal time step this becomes:

$$d\Pi = dV - \Delta dS \tag{1.12}$$

Substituting Equation 1.10 for dV and Equation 1.1 for dS we get.

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2})dt + \sigma S\frac{\partial V}{\partial S}dW_t - \Delta(rSdt + \sigma SdW_t) \\ &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2} - \Delta rS)dt + (\sigma S\frac{\partial V}{\partial S} - \Delta\sigma S)dW_t \end{aligned} \tag{1.13}$$

Since $\Delta = \frac{\partial V}{\partial S}$, Equation 1.13 becomes:

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS\frac{\partial V}{\partial S})dt + (\sigma S\frac{\partial V}{\partial S} - \sigma S\frac{\partial V}{\partial S})dW_t \\ &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS\frac{\partial V}{\partial S})dt \\ &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2})dt \end{aligned} \tag{1.14}$$

The **no-arbitrage condition** in a **risk-free portfolio** is:

$$d\Pi = r\Pi dt \tag{1.15}$$

Substituting this into Equation 1.14 we get:

$$\begin{aligned} r\Pi dt &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2})dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \end{aligned} \tag{1.16}$$

Substituting Equation 1.11 into Equation 1.16 we get:

$$\begin{aligned} r(V - \Delta S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\ r(V - \frac{\partial V}{\partial S}S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\ rV - rS\frac{\partial V}{\partial S} &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \end{aligned} \tag{1.17}$$

This gives us the final partial differential equation which is also the **Black-Scholes equation**:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0} \tag{1.18}$$

In this equation V is the option's price, t is the current time.
$\frac{\partial V}{\partial t}$ is the rate of change of the option's price to time, $\frac{\partial V}{\partial S}$ is the rate of change of the option's price to the underlying stock price and $\frac{\partial^2 V}{\partial S^2}$ is a concavity factor on how the option price changes with the stock price. The Black-Scholes can be solved for a call option and a put option. [3].

1.1.3 Closed-Form Solution for European Call Options

The closed-form solution for the Black-Scholes equation where the option is a call option is:

$$\begin{aligned} C(t,S) &= S\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \\ d_1 &= \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{1.19}$$

In this equation $T-t$ represents the time to maturity and K represents the strike price. [3].

$\Phi(z)$ is the standard normal cumulative distribution function which can be represented with the error function $\text{erf}(z)$:

$$\Phi(z) = \frac{1}{2}(1 + \text{erf}(\frac{z}{\sqrt{2}})) \tag{1.20}$$

In C++ this can be implemented as:

```cpp
#include <cmath>
#include <type_traits>

template <typename T>
auto normalCDF(T x) -> std::common_type_t<double, T>
{
    using commonType = std::common_type_t<double, T>;

    return static_cast<commonType>(0.5) * (static_cast<commonType>(1) +
    ↪  std::erf(x / std::sqrt(2.0)));
}
```

Listing 1.1: Normal cumulative distribution function

This function is defined in a header file called **functions.hpp** and the mechanism of this function is basically to return the result from Equation 1.19. Using the **cmath** header I compute the error function using **std::erf()**. This function returns a value with at least a type **double** depending on what type **x** is.

The next listing is a **struct** that is used throughout each engine that stores all of the data required to price a **Black Scholes option**. This struct is also in a header file called **black-scholes-struct.hpp**.

```
1   #ifndef GBMOPTIONS
2   #define GBMOPTIONS
3   #include <type_traits>
4
5   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
6   struct OptionDataBS
7   {
8       using CommonType = std::common_type_t <double, S_t, K_t, r_t,
        ↪  sigma_t, T_t>;
9       CommonType spot {};
10      CommonType strike {};
11      CommonType rate {};
12      CommonType volatility {};
13      CommonType maturity {};
14  };
15
16  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
17  OptionDataBS(S_t, K_t, r_t, sigma_t, T_t) -> OptionDataBS<S_t, K_t,
    ↪  r_t, sigma_t, T_t>;
18  #endif
```

Listing 1.2: Black Scholes Option Data Struct

Lines 1,2 and 18 are all part of a **header guard**. The struct **OptionDataBS** uses a template giving the opportunity for all of the data to have unique data types when passed into the struct. Line 8 creates a **type alias** which takes the common type of all of these parameters and sets it to something of at least type double. These types are then used to initiate the **spot, strike, rate, volatility** and **maturity** of this option. Line 16-17 is just a **class argument template deduction guide** so the compiler knows how to deduce the types of the arguments passed

Now to implement the function shown in Equation 1.19 in C++ we will use both the function and struct defined above:

```cpp
#include <cmath>
#include <type_traits>
#include "../Headers/functions.hpp"
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t, typename T_t>
auto blackScholesCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data) -> decltype(data.spot)
{
    using CommonType = decltype(data.spot);
    auto sqrt_maturity {std::sqrt(data.maturity)};

    auto d1 {(std::log(data.spot / data.strike) + (data.rate + (static_cast<CommonType>(0.5) * data.volatility * data.volatility)) * (data.maturity))/ (data.volatility * sqrt_maturity)};
    auto d2 {d1 - data.volatility * sqrt_maturity};

    auto C {data.spot * normalCDF(d1) - data.strike * std::exp(-data.rate * data.maturity) * normalCDF(d2)};
    return C;
}
```

Listing 1.3: Black Scholes Call using Closed Form Solution

The function in this listing is called **blackScholesCall**. This listing passes a reference to the struct **OptionDataBS** called **data** (since copying a struct is expensive) The return type of this function is deduced to be **commonType** which is also teh type of **data.spot** (hence I used **decltype**). The formula for d_1 is shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in Equation 1.19 is shown in line 15 using the previously defined function **normalCDF**. The function returns the value of the **call option** with at least type **double**.

1.1.4 Closed-Form Solution for European Put Options

The closed-form solution for the Black-Scholes equation fo ra put option is:

$$P(t, S) = Ke^{-r(T-t)}\Phi(-d_2) - S\Phi(-d_1) \tag{1.21}$$

using the same definitions of d_1 and d_2 in Equation 1.19 [3]

The implementation of this in C++ is:

```
1         #include <cmath>
2         #include <type_traits>
3         #include "../Headers/functions.hpp"
4         #include "../Headers/black_scholes_struct.hpp"
5
6         template <typename S_t, typename K_t, typename r_t, typename sigma_t,
          ↪  typename T_t>
7         auto blackScholesPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
          ↪  data) -> decltype(data.spot)
8         {
9             using CommonType = decltype(data.spot);
10            auto sqrt_maturity {std::sqrt(data.maturity)};
11
12            auto d1 {(std::log(data.spot / data.strike) + (data.rate +
              ↪  (static_cast<CommonType>(0.5) * data.volatility *
              ↪  data.volatility)) * (data.maturity)) / (data.volatility *
              ↪  sqrt_maturity)};
13            auto d2 {d1 - data.volatility * sqrt_maturity};
14
15            auto P  {data.strike * std::exp(-data.rate * data.maturity) *
              ↪  normalCDF(-d2) - data.spot * normalCDF(-d1)};
16            return P;
17        }
18
```

Listing 1.4: Black Scholes Put using Closed Form Solution

The function in this listing is called **blackScholesPut**. This listing is almost identical to the one for **blackScholesCall**. The formula for d_1 is again shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in Equation 1.21 is computed in line 15 (stored as **P**) using the function **normalCDF**. The function returns the value of the **put option** with at least type **double**.

1.2 Black-76 PDE

The Black-76 model is just an adaption of the **Black-Scholes equation** except it uses the price of futures and bonds to price options.
A **future** is a contract where the buyer/seller agrees to purchase/sell an asset at a later fixed time

In **GitHub** the source file can be found here: **source file**

For the listings in this section I will continue using the header file **functions.hpp** that I defined on day 1. I will continue to use the **normalCDF** function from there.

1.2.1 Derivation of the Black-76 PDE

The modelling assumptions are that the futures of forward price process is modelled by **geometric Brownian motion** shown by the equation:

$$dF_t = \sigma F_t dW_t \tag{1.22}$$

where F_t is the price of a future or a forward at a given time, dF_t is the infinitesimal change of the price of a future of a forward, σ is implied volatility and dW_t is the infinitesimal change in the **Wiener function**.[4]

Substituting Equation 1.22 into **Ito's Lemma** 1.3

$$\begin{aligned} df(t,S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial F_t}(\sigma F_t dW_t) + \frac{1}{2}\frac{\partial^2 f}{\partial F_t^2}(\sigma F_t dW_t)^2 \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial F_t}(\sigma F_t dW_t) + \frac{1}{2}\frac{\partial^2 f}{\partial F_t^2}(\sigma^2 F_t^2 (dW_t)^2) \end{aligned} \tag{1.23}$$

Substituting Equations 1.7 into Equation 1.23 we get:

$$\begin{aligned} df(t,S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial F_t}(\sigma F_t)dW_t + \frac{1}{2}\frac{\partial^2 f}{\partial F_t^2}(\sigma^2 F_t^2)dt \\ &= (\frac{\partial f}{\partial t} + \frac{1}{2}\sigma^2 F_t^2 \frac{\partial^2 f}{\partial F_t^2})dt + \sigma F_t \frac{\partial f}{\partial F_t}dW_t \end{aligned} \tag{1.24}$$

Define

$$V(t,S) \equiv f(t,S_t) \tag{1.25}$$

Using the result from Equation 1.24

$$dV(t,S) = (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 V}{\partial F^2})dt + \sigma F \frac{\partial V}{\partial F} dW_t \tag{1.26}$$

The thing that is different about the **delta-hedged portfolio** is that at the time the option is being exchanged, the cost of the **future** is 0.

$$\Pi = V \tag{1.27}$$

For an infinitesimal time step, the value of the future increases by dF which leads to

$$d\Pi = dV - \Delta dF \tag{1.28}$$

where $\Delta = \frac{\partial V}{\partial F}$ Substituting Equation 1.26 for dV and Equation 1.22 for dF we get.

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2})dt + \sigma F \frac{\partial V}{\partial F} dW_t - \Delta(\sigma F dW_t) \\ &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2})dt + (\sigma F \frac{\partial V}{\partial F} - \Delta\sigma F)dW_t \end{aligned} \tag{1.29}$$

Since $\Delta = \frac{\partial V}{\partial F}$, Equation 1.29 becomes:

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2})dt + (\sigma F \frac{\partial V}{\partial F} - \sigma F \frac{\partial V}{\partial F})dW_t \\ &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2})dt \end{aligned} \tag{1.30}$$

Using Equation 1.15 and substituting this into Equation 1.30 we get:

$$\begin{aligned} r\Pi dt &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2})dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \end{aligned} \tag{1.31}$$

Substituting Equation 1.27 into Equation 1.31 we get:

$$rV = \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \tag{1.32}$$

Which finally leads to the equation:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} - rV = 0} \tag{1.33}$$

Which is also known as the **Black-76 Equation**

1.2.2 Implementation of the Black-76 Struct

Just like the Black-Scholes equation, the **Black-76** equation has its own struct since it uses **futures/forwards** instead of **stocks**. I will need to save the initial value of the future instead of a stock.

```
#ifndef BLACK76
#define BLACK76
#include <type_traits>

template <typename F_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>

struct OptionDataB76
{
    using commonType = std::common_type_t <double, F_t, K_t, r_t,
    ↪  sigma_t, T_t>;
    commonType future {};
    commonType strike {};
    commonType rate {};
    commonType volatility {};
    commonType maturity {};
};

template <typename F_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
OptionDataB76(F_t, K_t, r_t, sigma_t, T_t) -> OptionDataB76<F_t, K_t,
↪  r_t, sigma_t, T_t>;

#endif


```

Listing 1.5: Black 76 Option Data Struct

This struct **OptionDataB76** is almost the same as the struct **OptionDataBS** except it passes a future instead of a stock. This means the type placeholder also changed from `S_t` for stocks to `F_t` for futures. The first data member the struct saves is **future** instead of **spot**. This struct is saved on the header: **black76-structs.hpp**

1.2.3 Black-76 Formula for Call Options

The Black-76 Formula for Call options is:

$$\begin{aligned} C(t,F) &= e^{-r(T-t)}(F\Phi(d_1) - K\Phi(d_2)) \\ d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{1.34}$$

[4] The implementation in C++ is:

```
1   #include <cmath>
2   #include <type_traits>
3   #include "../Headers/functions.hpp"
4   #include "../Headers/black76_structs.hpp"
5
6   template <typename F_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
7   auto black76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
    ↪  data) -> decltype(data.future)
8   {
9       using commonType = decltype(data.future);
10      auto sqrt_maturity {std::sqrt(data.maturity)};
11
12      auto d1 {(std::log(data.future / data.strike) +
        ↪  (static_cast<commonType>(0.5) * data.volatility *
        ↪  data.volatility) * data.maturity ) / (data.volatility*
        ↪  sqrt_maturity)};
13      auto d2 {d1 - data.volatility * sqrt_maturity};
14
15      auto C {(std::exp(-data.rate * data.maturity)) * (data.future *
        ↪  normalCDF(d1) - data.strike * normalCDF(d2))};
16      return C;
17  }
```

Listing 1.6: Black-76 Call using Closed Form Solution

d_1 is calculated in line 12, d_2 is calculated in line 13 and finally Equation 1.34 is computed in line 15 (stored as **C**) and returned in line 16.

1.2.4 Black-76 Formula for Put Options

The Black-76 Formula for Call options is:

$$\begin{aligned} P(t,F) &= e^{-r(T-t)}(K\Phi(-d_2) - F\Phi(-d_1)) \\ d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{1.35}$$

[4] The implementation in C++ is:

```
1   #include <cmath>
2   #include <type_traits>
3   #include "../Headers/functions.hpp"
4   #include "../Headers/black76_structs.hpp"
5
6   template <typename F_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
7   auto black76Put(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>& data)
    ↪  -> decltype(data.future)
8   {
9       using commonType = decltype(data.future);
10      auto sqrt_maturity {std::sqrt(data.maturity)};
11
12      auto d1 {(std::log(data.future / data.strike) +
        ↪  (static_cast<commonType>(0.5) * data.volatility *
        ↪  data.volatility) * data.maturity ) / (data.volatility*
        ↪  sqrt_maturity)};
13      auto d2 {d1 - data.volatility * sqrt_maturity};
14
15      auto P {std::exp(-data.rate * data.maturity) * (data.strike *
        ↪  normalCDF(-d2) - (data.future * normalCDF(-d1)))};
16      return P;
17  }
```

Listing 1.7: Black-76 Call using Closed Form Solution

d_1 is calculated in line 12, d_2 is calculated in line 13 and finally Equation 1.35 is computed in line 15 (stored as **P**) and returned in line 16.

1.3 Merton's Dividend-Adjusted Black-Scholes

A **dividend** is cash payment from companies to shareholders for ownership in their stock. The key property with dividends is that each time a dividend is given to the shareholder, the stock price per share drops by that dividend amount and that the dividend grows as the stock grows.

The first subsubsection is a derivation of the **Merton's Dividend Adjust Black-Scholes PDE**, followed by two subsubections on the closed form of the call option and on the closed form of the put option.

Both of these have my implementations in **C++**. In **GitHub** the source file can be found here: source file
as well as the header file: functions.hpp

1.3.1 Derivation of Merton's Dividend-Adjusted Black-Scholes

Let's define the dividend yield to be q. For each infinitesimal step in the stock price S_t we pay a dividend of value $qSdt$. Putting this into the adjusted **geometric Brownian motion** give us:

$$\begin{aligned} dS_t &= rS_t dt + \sigma S_t dW_t - qS_t dt \\ &= (r-q)S_t dt + \sigma S_t dW_t \end{aligned} \tag{1.36}$$

where the other variables defined here are the same defined in Equation 1.1. The derivation here is again almost identical to that of the **Black-Scholes equation**. Substituting Equation 1.36 for dS_t we get:

$$\begin{aligned} df(t,S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left((r-q)S_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left((r-q)S_t dt + \sigma S_t dW_t\right)^2 \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left((r-q)S_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left((r-q)^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 (dt)(dW_t) + \sigma^2 S_t^2 (dW_t)^2\right) \end{aligned} \tag{1.37}$$

Substituting Equations 1.5, 1.6, and 1.7 into Equation 1.37 we get:

$$\begin{aligned} df(t,S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left((r-q)S_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(0 + 0 + \sigma^2 S_t^2 (dt)\right) \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left((r-q)S_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\sigma^2 S_t^2 \\ &= \left(\frac{\partial f}{\partial t} + (r-q)S_t\frac{\partial f}{\partial S_t} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 f}{\partial S_t^2}\right)dt + \sigma S_t\frac{\partial f}{\partial S_t}dW_t \end{aligned} \tag{1.38}$$

Define

$$V(t,S) \equiv f(t,S_t) \tag{1.39}$$

Using the result from Equation 1.38

$$dV(t,S) = (\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 V}{\partial S^2})dt + \sigma S \frac{\partial V}{\partial S} dW_t \tag{1.40}$$

Now we will define a delta-hedged portfolio Π and $\Delta = \frac{\partial V}{\partial S}$ that satisfies:

$$\Pi = V - \Delta S \tag{1.41}$$

For an infinitesimal time step, since we hedge the option by shorting the stock, we must issue dividends with yield q which is represented in an infinitesimal time step as: $-\Delta q S dt$ this becomes:

$$d\Pi = dV - \Delta dS - \Delta q S dt \tag{1.42}$$

Substituting Equation 1.40 for dV and Equation 1.1 for dS we get.

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 V}{\partial S^2})dt + \sigma S \frac{\partial V}{\partial S} dW_t - \Delta((r-q)Sdt + \sigma S dW_t) - \Delta q S dt \\ &= (\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 V}{\partial S^2} - \Delta(r-q)S - \Delta q S)dt + (\sigma S \frac{\partial V}{\partial S} - \Delta \sigma S)dW_t \end{aligned} \tag{1.43}$$

Since $\Delta = \frac{\partial V}{\partial S}$, Equation 1.43 becomes:

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - (r-q)S\frac{\partial V}{\partial S} - qS\frac{\partial V}{\partial S})dt + (\sigma S \frac{\partial V}{\partial S} - \sigma S \frac{\partial V}{\partial S})dW_t \\ &= (\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - (r-q)S\frac{\partial V}{\partial S} - qS\frac{\partial V}{\partial S})dt \\ &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S})dt \end{aligned} \tag{1.44}$$

Substituting Equation 1.15 intoEquation 1.44 we get:

$$\begin{aligned} r\Pi dt &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S})dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S} \end{aligned} \tag{1.45}$$

Substituting Equation 1.41 into Equation 1.45 we get:

$$\begin{aligned} r(V - \Delta S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S} \\ r(V - \frac{\partial V}{\partial S}S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S} \\ rV - rS\frac{\partial V}{\partial S} &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS\frac{\partial V}{\partial S} \end{aligned} \tag{1.46}$$

This finally gives us the equation:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r-q)S\frac{\partial V}{\partial S} - rV = 0} \tag{1.47}$$

which is the **Black-Scholes PDE with Merton's dividend adjustment**. [5]

This PDE uses the same variables as the **Black-Scholes PDE** with the only addition being the **dividend yield** q. q affects the equaton by modifying $rS\frac{\partial V}{\partial S}$ (Black-Scholes PDE) into the new $(r-q)S\frac{\partial V}{\partial S}$

1.3.2 Implementation of the Black-Scholes Merton Struct

```
#include <type_traits>
#ifndef MERTONSTRUCTS
#define MERTONSTRUCTS

template <typename S_t, typename K_t, typename r_t, typename q_t,
↪  typename sigma_t, typename T_t>

struct OptionDataMerton
{
    using CommonType = std::common_type_t <double, S_t, K_t, r_t, q_t,
    ↪  sigma_t, T_t>;
    CommonType spot {};
    CommonType strike {};
    CommonType rate {};
    CommonType dividend {};
    CommonType volatility {};
    CommonType maturity {};
};

template <typename S_t, typename K_t, typename r_t, typename q_t,
↪  typename sigma_t, typename T_t>
OptionDataMerton(S_t, K_t, r_t, q_t, sigma_t, T_t) ->
↪  OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t, T_t>;
#endif
```

Listing 1.8: Black–Scholes Merton Option Data Struct

This struct **OptionDataMerton** is very similar to both of the ones above, however we now also have the dividend member. The type placeholder for the dividend is `q_t` and it is initialised right after rate. This struct can be found in the header file: **merton-struct.hpp**

1.3.3 Closed-Form Solution to Call Options with Dividends

$$
\begin{aligned}
C(t,S) &= Se^{-q(T-t)}\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \\
d_1 &= \frac{\ln(\frac{S}{K}) + (r - q + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \\
d_2 &= d_1 - \sigma\sqrt{T-t}
\end{aligned}
\tag{1.48}
$$

[5] The only thing that changed from Equation 1.19 is the d_1 term has a $-q$ term as well as the fact that in $C(t, S)$, the S term has an $e^{-q(T-t)}$ term multiplied to it.

Implementation in C++

```
1   #include <cmath>
2   #include <type_traits>
3   #include "../Headers/functions.hpp"
4   #include "../Headers/merton_struct.hpp"
5
6   template <typename S_t, typename K_t, typename r_t, typename q_t,
    ↪  typename sigma_t, typename T_t>
7   auto bsDividendCall(const OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t,
    ↪  T_t>& data) -> decltype(data.spot)
8   {
9       using CommonType = decltype(data.spot);
10      auto sqrt_maturity {std::sqrt(data.maturity)};
11
12      auto d1 {(std::log(data.spot / data.strike) + (data.rate  -
        ↪  data.dividend + (static_cast<CommonType>(0.5) * data.volatility
        ↪  * data.volatility)) * (data.maturity))/ (data.volatility *
        ↪  sqrt_maturity)};
13      auto d2 {d1 - data.volatility * sqrt_maturity};
14
15      auto C {data.spot * std::exp(-data.dividend * data.maturity) *
        ↪  normalCDF(d1) - data.strike * std::exp(-data.rate *
        ↪  data.maturity) * normalCDF(d2)};
16      return C;
17  }
```

Listing 1.9: Black-76 Call using Closed Form Solution

This function is similar to the call function made in day 1, however the struct **OptionDataMerton** also has the data member **dividend**. The addition of **data.dividend** affects line 12 with the $(r - q)$ term instead of r. The addition also affects line 15 where the value of S is affected with $Se^{-q(T-t)}$ versus S. This function evaluates Equation 1.48 and stores it as **C** with a type of at least **double** and then returns it in line 16.

1.3.4 Closed-Form Solution to Put Options with Dividends

$$
\begin{aligned}
P(t,S) &= Ke^{-r(T-t)}\Phi(-d_2) - Se^{-q(T-t)}\Phi(-d_1) - \\
d_1 &= \frac{\ln(\frac{S}{K}) + (r - q + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \\
d_2 &= d_1 - \sigma\sqrt{T-t}
\end{aligned}
\tag{1.49}
$$

[5] Again just like with the call, the only changes from Equation 1.21 is the fact that the d_1 term has a $-q$ term as well as the fact that in $P(t,S)$, the S term has an $e^{-q(T-t)}$ term multiplied to it.

Implementation in C++

```
1        #include <cmath>
2        #include <type_traits>
3        #include "../Headers/functions.hpp"
4        #include "../Headers/merton_struct.hpp"
5
6        template <typename S_t, typename K_t, typename r_t, typename q_t,
         ↪  typename sigma_t, typename T_t>
7        auto bsDividendPut(const OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t,
         ↪  T_t>& data) -> decltype(data.spot)
8        {
9            using CommonType = decltype(data.spot);
10           auto sqrt_maturity {std::sqrt(data.maturity)};
11
12           auto d1 {(std::log(data.spot / data.strike) + (data.rate -
             ↪  data.dividend + (static_cast<CommonType>(0.5) * data.volatility
             ↪  * data.volatility)) * (data.maturity)) / (data.volatility *
             ↪  sqrt_maturity)};
13           auto d2 {d1 - data.volatility * sqrt_maturity};
14
15           auto P  {data.strike * std::exp(-data.rate * data.maturity) *
             ↪  normalCDF(-d2) - data.spot * std::exp(-data.dividend *
             ↪  data.maturity) * normalCDF(-d1)};
16           return P;
17       }
```

Listing 1.10: Black-76 Call using Closed Form Solution

Function got the same changes as in listing 1.9 where it calculates d_1 in line 12, d_2 in line 13 and computes Equation 1.49 in line 15 and returns it in line 18.

2 Monte Carlo Simulations

2.1 Monte Carlo Simulation with the GBM Assumption

The source file can be found on my **GitHub**

2.1.1 Derivation of the value of a Stock

First we will use the **Geometric Brownian Motion** assumption defined in Equation 1.1

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t$$
$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t$$

Define:

$$X_t = \ln(S_t) \tag{2.1}$$

Using **Ito's Lemma** we can represent dX_t in terms of dS_t like this:

$$dX_t = \frac{dX_t}{dS_t}\,dS_t + \frac{1}{2}\frac{d^2X_t}{dS_t^2}(dS_t)^2$$

Since $\frac{dX_t}{dS_t} = \frac{1}{S_t}$ and $\frac{d^2X_t}{dS_t^2} = -\frac{1}{(S_t)^2}$. Substituting this in the previous equation we get:

$$\begin{aligned} dX_t &= \frac{1}{S_t}\left(rS_t\,dt + \sigma S_t\,dW_t\right) - \frac{1}{2S_t^2}\left(rS_t\,dt + \sigma S_t\,dW_t\right)^2 \\ dX_t &= r\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left(r^2S_t^2(dt)^2 + 2r\sigma S_t^2\,dt\,dW_t + \sigma^2S_t^2(dW_t)^2\right) \end{aligned} \tag{2.2}$$

Using **Ito's multiplication rules** defined in Equation 1.7, we can simplify the equation to:

$$\begin{aligned} dX_t &= r\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left(\sigma^2S_t^2\,dt\right) \\ dX_t &= \left(r - \frac{1}{2}\sigma^2\right)dt + \sigma\,dW_t \end{aligned} \tag{2.3}$$

Introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned} \int_t^T dX_s &= \int_t^T \left(r - \frac{1}{2}\sigma^2\right)ds + \int_t^T \sigma\,dW_s \\ X_T - X_t &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \end{aligned} \tag{2.4}$$

Substituting $ln(S_T) = X_T$ and $ln(S_t) = X_t$ into Equation 2.24 we get:

$$\begin{aligned} \ln(S_T) - \ln(S_t) &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \ln\left(\frac{S_T}{S_t}\right) &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \frac{S_T}{S_t} &= \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right) \end{aligned} \tag{2.5}$$

This leaves us with the equation of a stock which is:

$$S_T = S_t \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right) \tag{2.6}$$

2.1.2 Monte Carlo Simulations for BS Call Options

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(S_T - K, 0) \tag{2.7}$$

Where S_T is the stock price at the time of maturity T

When we use a Monte Carlo Simulation we simulate a lot of these values for S_T to find the expected value of a payoff. After we find the expected value of a payoff, we must discount it using the risk-free rate r like this:

$$C = e^{-r(T-t)}\mathbb{E}(\max(S_T - K, 0)) \tag{2.8}$$

where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.6 In order to simulate the Wiener function, we use Equation 1.2

```cpp
#include <cmath>
#include <random>
#include <algorithm>
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t, typename T_t>
auto monteCarloBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);
    std::random_device rd{};
    std::mt19937 mt{rd()};
    std::normal_distribution W {static_cast<commonType>(0), std::sqrt(data.maturity)};
    commonType payoffs {};

    for (int i {}; i < simulations; ++i)
    {
        commonType S_final {data.spot * std::exp((data.rate - 0.5 * data.volatility * data.volatility) * data.maturity + (data.volatility * W(mt)))};
        payoffs += std::max(S_final - data.strike, static_cast<commonType>(0));
    }
    payoffs /= simulations;

    return std::exp(-data.rate * data.maturity) * payoffs;
}
```

Listing 2.1: Black-Scholes Call using Monte Carlo Simulations

First, this function takes in the struct defined in Listing 1.2 as well as another parameter called **simulations**.
In line 10 we define a random device in order to seed our random number generator. For the random number generator in line 11 we use a **Mersenne Twister** generator. Line 12 uses the **std::normal distribution** function from the **cmath** header that will normalise the random variable we generate into a standard normal random variable as shown in Equation 1.2. In line 15 a **for loop** is made with **simulation** number of iterations.
Line 17 it computes Equation 2.6 and then is used in the Equation 2.7 which is computed in line 18.
After the for loop ends, an average of the payoffs is taken in line 20 by dividing the

total payoffs by the number of simulations, and finally in line 22 this average payoff is discounted and then returned as the call price.

2.1.3 Monte Carlo Simulations for BS Put Options

For a put option, it is pretty much the opposite, the payoff for a put option is defined as:

$$\text{Payoff} = \max(K - S_T, 0) \tag{2.9}$$

The equation for the put option value at time t is identical to that of a call option, just the payoff is different. The value of a put option is represented as:

$$P = e^{-r(T-t)}\mathbb{E}(\max(K - S_T, 0)) \tag{2.10}$$

where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.6

```cpp
#include <cmath>
#include <random>
#include <algorithm>
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t, typename T_t>
auto monteCarloBSPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);
    std::random_device rd{};
    std::mt19937 mt{rd()};
    std::normal_distribution W {static_cast<commonType>(0), std::sqrt(data.maturity)};
    commonType payoffs {};

    for (int i {}; i < simulations; ++i)
    {
        commonType S_final {data.spot * std::exp((data.rate - 0.5 * data.volatility * data.volatility) * data.maturity + (data.volatility * W(mt)))};
        payoffs += std::max(data.strike - S_final , static_cast<commonType>(0));
    }
    payoffs /= simulations;

    return std::exp(-data.rate * data.maturity) * payoffs;
}
```

Listing 2.2: Black-Scholes Put using Monte Carlo Simulations

This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.9 instead of Equation 2.7

2.2 Monte Carlo Simulation for Black 76

The source file can be found on my **GitHub**

2.2.1 Derivation of the value of a Future

First we will use the Black-76 assumption defined in Equation 1.22

$$\begin{aligned} dF_t &= \sigma F_t \, dW_t \\ \frac{dF_t}{F_t} &= \sigma \, dW_t \end{aligned}$$

Define:

$$X_t = \ln(F_t) \tag{2.11}$$

Using **Ito's Lemma** we can represent dX_t in terms of dF_t like this:

$$dX_t = \frac{dX_t}{dF_t} \, dF_t + \frac{1}{2} \frac{d^2 X_t}{dF_t^2} (dF_t)^2$$

Since $\frac{dX_t}{dF_t} = \frac{1}{F_t}$ and $\frac{d^2 X_t}{dF_t^2} = -\frac{1}{(F_t)^2}$. Substituting this in the previous equation we get:

$$dX_t = \frac{1}{F_t} \left(\sigma F_t \, dW_t\right) - \frac{1}{2F_t^2} \left(\sigma F_t \, dW_t\right)^2 \tag{2.12}$$

Using **Ito's multiplication rules** defined in Equation 1.5 , 1.6 and 1.7, we can simplify the equation to:

$$\begin{aligned} dX_t &= \sigma \, dW_t - \frac{1}{2F_t^2} \left(\sigma^2 F_t^2 \, dt\right) \\ dX_t &= \sigma \, dW_t - \frac{1}{2} \sigma^2 dt \end{aligned} \tag{2.13}$$

Again by introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned} \int_t^T dX_s &= \int_t^T \sigma \, dW_s - \int_t^T \frac{1}{2} \sigma^2 ds \\ X_T - X_t &= \sigma (W_T - W_t) - \frac{1}{2} \sigma (T - t) \end{aligned} \tag{2.14}$$

Substituting $ln(F_T) = X_T$ and $ln(F_t) = X_t$ into Equation 2.14 we get:

$$\begin{aligned} \ln(F_T) - \ln(F_t) &= \sigma(W_T - W_t) - \frac{1}{2}\sigma(T-t) \\ \ln\left(\frac{F_T}{F_t}\right) &= \sigma(W_T - W_t) - \frac{1}{2}\sigma(T-t) \\ \frac{F_T}{F_t} &= \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma(T-t)) \end{aligned} \tag{2.15}$$

This leaves us with the equation of a stock which is:

$$F_T = F_t \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma(T-t)) \tag{2.16}$$

2.2.2 Monte Carlo Simulations for B76 Call Options

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(F_T - K, 0) \tag{2.17}$$

Where S_T is the stock price at the time of maturity T

When we use a Monte Carlo Simulation we simulate a lot of these values for S_T to find the expected value of a payoff. After we find the expected value of a payoff, we must discount it using the risk-free rate r like this:

$$C = e^{-r(T-t)}\mathbb{E}(\max(F_T - K, 0)) \tag{2.18}$$

where $e^{-r(T-t)}$ is the discount factor and F_T is the same as defined in Equation 2.16 In order to simulate the Wiener function, we use Equation 1.2

```cpp
#include <cmath>
#include <random>
#include <algorithm>
#include "../Headers/black76_structs.hpp"

template <typename F_t, typename K_t, typename r_t, typename sigma_t, typename T_t>
auto monteCarloB76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>& data, int simulations) -> decltype(data.future)
{
    using commonType = decltype(data.future);
    std::random_device rd{};
    std::mt19937 mt{rd()};
    std::normal_distribution W {static_cast<commonType>(0), std::sqrt(data.maturity)};
    commonType payoffs {};

    for (int i {}; i < simulations; ++i)
    {
        commonType F_final {data.future * std::exp(((data.volatility * W(mt)) - 0.5 * data.volatility * data.volatility * data.maturity))};
        payoffs += std::max(F_final - data.strike, static_cast<commonType>(0));
    }
    payoffs /= simulations;

    return std::exp(-data.rate * data.maturity) * payoffs;
}
```

Listing 2.3: Black-76 Call using Monte Carlo Simulations

This function is really similar to that of the one defined in Listing 2.2 when it comes to the random number generation. This function passes in the **OptionDataB76** Struct instead as well as the integer parameter **simulations**. It computes Equation **??** in line 18 and takes the average, discounts it and returns the value of the call option in line 22.

2.2.3 Monte Carlo Simulations for B76 Put Options

For a put option, it is pretty much the opposite, the payoff for a put option is defined as:

$$\text{Payoff} = \max(K - F_T, 0) \tag{2.19}$$

The equation for the put option value at time t is represented as:

$$P = e^{-r(T-t)}\mathbb{E}(\max(K - F_T, 0)) \tag{2.20}$$

where $e^{-r(T-t)}$ is the discount factor and F_T is the same as defined in Equation 2.16

```
1   #include <cmath>
2   #include <random>
3   #include <algorithm>
4   #include "../Headers/black76_structs.hpp"
5
6   template <typename F_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
7   auto monteCarloB76Put(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
    ↪  data, int simulations) -> decltype(data.future)
8   {
9       using commonType = decltype(data.future);
10      std::random_device rd{};
11      std::mt19937 mt{rd()};
12      std::normal_distribution W {static_cast<commonType>(0),
        ↪  std::sqrt(data.maturity)};
13      commonType payoffs {};
14
15      for (int i {}; i < simulations; ++i)
16      {
17          commonType F_final {data.future * std::exp(((data.volatility *
            ↪  W(mt)) - 0.5 * data.volatility * data.volatility *
            ↪  data.maturity))};
18          payoffs += std::max(data.strike - F_final,
            ↪  static_cast<commonType>(0));
19      }
20      payoffs /= simulations;
21
22      return std::exp(-data.rate * data.maturity) * payoffs;
23  }
```

Listing 2.4: Black-76 Put using Monte Carlo Simulations

This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.19 instead of Equation 2.17

2.3 Monte Carlo Simulation with Dividends

The source can again be found on my **GitHub**

2.3.1 Derivation of the value of a Stock with Dividends

First we will use the **Merton SDE** defined in Equation 1.36

$$\begin{aligned} dS_t &= (r-q)S_t\,dt + \sigma S_t\,dW_t \\ \frac{dS_t}{S_t} &= (r-q)\,dt + \sigma\,dW_t \end{aligned}$$

Define:

$$X_t = \ln(S_t) \tag{2.21}$$

Using **Ito's Lemma** we can represent dX_t in terms of dS_t like this:

$$dX_t = \frac{dX_t}{dS_t}\,dS_t + \frac{1}{2}\frac{d^2X_t}{dS_t^2}(dS_t)^2$$

Since $\frac{dX_t}{dS_t} = \frac{1}{S_t}$ and $\frac{d^2X_t}{dS_t^2} = -\frac{1}{(S_t)^2}$. Substituting this in the previous equation we get:

$$\begin{aligned} dX_t &= \frac{1}{S_t}\left((r-q)S_t\,dt + \sigma S_t\,dW_t\right) - \frac{1}{2S_t^2}\left((r-q)S_t\,dt + \sigma S_t\,dW_t\right)^2 \\ dX_t &= (r-q)\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left((r-q)^2S_t^2(dt)^2 + 2r\sigma S_t^2\,dt\,dW_t + \sigma^2S_t^2(dW_t)^2\right) \end{aligned} \tag{2.22}$$

Using **Ito's multiplication rules** defined in Equation 1.5, 1.6, and 1.7, we can simplify the equation to:

$$\begin{aligned} dX_t &= (r-q)\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left(\sigma^2S_t^2\,dt\right) \\ dX_t &= \left(r - q - \frac{1}{2}\sigma^2\right)dt + \sigma\,dW_t \end{aligned} \tag{2.23}$$

Introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned} \int_t^T dX_s &= \int_t^T \left(r - q - \frac{1}{2}\sigma^2\right)ds + \int_t^T \sigma\,dW_s \\ X_T - X_t &= \left(r - q - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \end{aligned} \tag{2.24}$$

Substituting $ln(S_T) = X_T$ and $ln(S_t) = X_t$ into Equation 2.24 we get:

$$\begin{aligned} \ln(S_T) - \ln(S_t) &= \left(r - q - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \ln\left(\frac{S_T}{S_t}\right) &= \left(r - q - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \frac{S_T}{S_t} &= \exp\left(\left(r - q - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right) \end{aligned} \tag{2.25}$$

This leaves us with the equation of a stock which is:

$$S_T = S_t \exp\left(\left(r - q - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right) \tag{2.26}$$

2.3.2 Monte Carlo Simulations for Call Options with Dividends

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(S_T - K, 0) \tag{2.27}$$

Where S_T is the stock price with dividends at the time of maturity T.

The value of the call option is equal to:

$$C = e^{-r(T-t)}\mathbb{E}(\max(S_T - K, 0)) \tag{2.28}$$

where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.26

Implementing the call option pricer in C++ like this:

```
1   #include <cmath>
2   #include <random>
3   #include <algorithm>
4   #include "../Headers/merton_struct.hpp"
5
6   template <typename S_t, typename K_t, typename r_t, typename q_t,
    ↪  typename sigma_t, typename T_t>
7   auto monteCarloMertonCall(const OptionDataMerton<S_t, K_t, r_t, q_t,
    ↪  sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
8   {
9       using commonType = decltype(data.spot);
10      std::random_device rd{};
11      std::mt19937 mt{rd()};
12      std::normal_distribution W {static_cast<commonType>(0),
        ↪  std::sqrt(data.maturity)};
13      commonType payoffs {};
14
15      for (int i {}; i < simulations; ++i)
16      {
17          commonType S_final {data.spot * std::exp((data.rate -
            ↪  data.dividend - 0.5 * data.volatility * data.volatility) *
            ↪  data.maturity + (data.volatility * W(mt)))};
18          payoffs += std::max(S_final - data.strike,
            ↪  static_cast<commonType>(0));
19      }
20      payoffs /= simulations;
21
22      return std::exp(-data.rate * data.maturity) * payoffs;
23  }
```

Listing 2.5: Black-Scholes -Merton Call using Monte Carlo Simulations

This function **monteCarloMertonCall** is extremely similar to the one from the GBM Monte Carlo simulation except for the fact it introduces another parameter in the struct **OptionDataMerton** which is the data member **dividend**.
This modified line 18 with the modification to Equation 2.27 which then influences the value being returned in line 22 representing Equation 2.28.

2.3.3 Monte Carlo Simulations for Put Options with Dividends

For a put option, it again is pretty much the opposite, with the payoff for a put option being defined as:

$$\text{Payoff} = \max(K - S_T, 0) \tag{2.29}$$

The value of a put option is represented as:

$$P = e^{-r(T-t)}\mathbb{E}(\max(K - S_T, 0)) \tag{2.30}$$

where S_T is the same as defined in Equation 2.26

```cpp
#include <cmath>
#include <random>
#include <algorithm>
#include "../Headers/merton_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename q_t,
    typename sigma_t, typename T_t>
auto monteCarloMertonPut(const OptionDataMerton<S_t, K_t, r_t, q_t,
    sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);
    std::random_device rd{};
    std::mt19937 mt{rd()};
    std::normal_distribution W {static_cast<commonType>(0),
        std::sqrt(data.maturity)};
    commonType payoffs {};

    for (int i {}; i < simulations; ++i)
    {
        commonType S_final {data.spot * std::exp((data.rate -
            data.dividend - 0.5 * data.volatility * data.volatility) *
            data.maturity + (data.volatility * W(mt)))};
        payoffs += std::max(data.strike - S_final,
            static_cast<commonType>(0));
    }
    payoffs /= simulations;

    return std::exp(-data.rate * data.maturity) * payoffs;
}
```

Listing 2.6: Black-Scholes-Merton Put using Monte Carlo Simulations

This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.29 instead of Equation 2.27

2.4 Monte Carlo Variance Reduction Strategies

2.4.1 Convergence without Variance Reduction Strategies

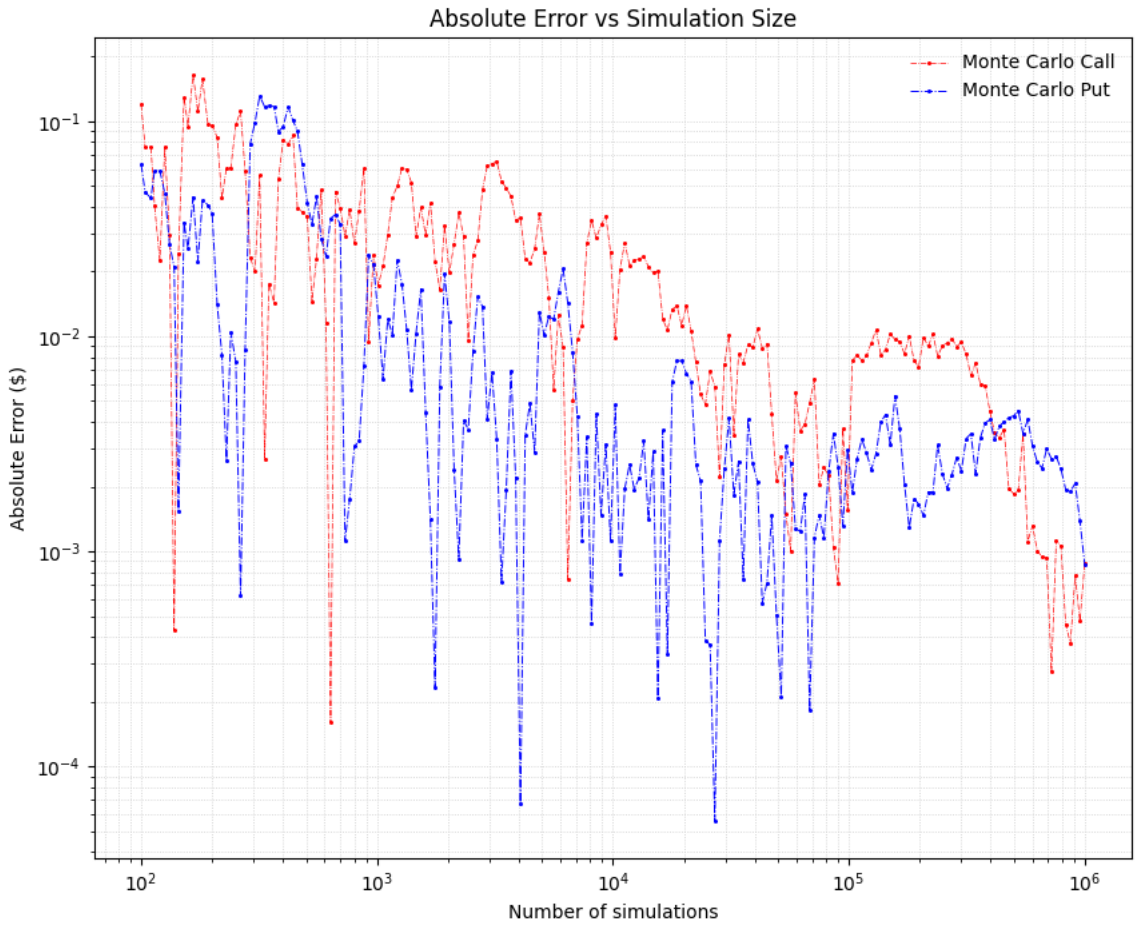


Figure 1: Absolute error against the number of Monte Carlo simulations.

2.4.2 Antithetic Variates

3 Binomial Trees

3.1 Binomial Trees with the GBM Assumption

3.1.1 Deriving the Risk-Neutral Probability

A Binomial Tree is a numerical method that is produced by creating values until the time to maturity. The nodes at maturity are all the payoffs, and with the binomial tree we work backwards to find the price of the option at the initial time.

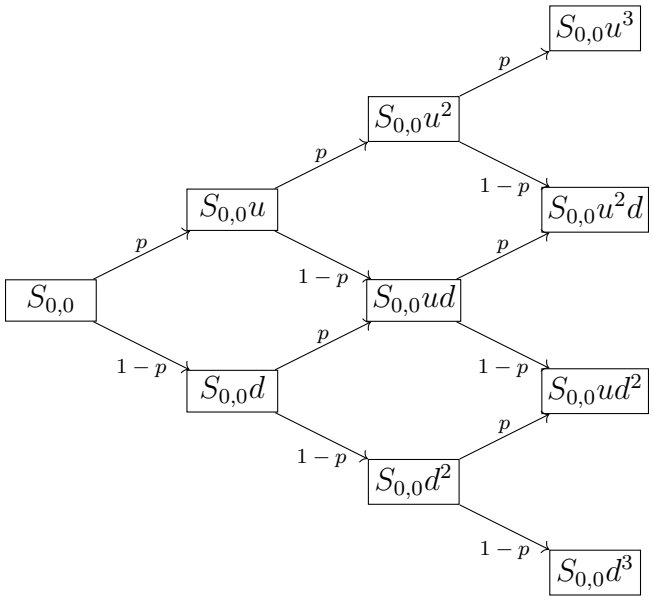


Figure 2: Binomial Tree for the underlying asset price

This is what a binomial tree for a stock looks like. It can go up with probability p, and go down with probability $1-p$. The stock $S_{a,b}$ is the node that is up a nodes and down b nodes

$$S_{a,b} = u^a d^b S_{0,0} \tag{3.1}$$

In order to derive the values for u and d we are going to use the equation of a stock

from Equation 2.6

$$S_T = S_t \exp((r - \frac{1}{2}\sigma^2)(T-t) + \sigma(W_T - W_t))$$

On an interval of time Δt we have

$$\frac{S_{t+\Delta t}}{S_t} = \frac{S_t \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t))}{S_t}$$
$$\frac{S_{t+\Delta t}}{S_t} = \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t))$$

Using equation 1.2 we have:

$$W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$$
$$W_{t+\Delta t} - W_t \sim \Delta t(\mathcal{N}(0,1))$$

Letting $Z \sim \mathcal{N}(0,1)$ we have:

$$\frac{S_{t+\Delta t}}{S_t} = \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma\sqrt{\Delta t}Z$$

At very small Δt , $\sqrt{\Delta t} \gg \Delta t$ which means this equation can be approximated to:

$$\frac{S_{t+\Delta t}}{S_t} \approxeq \exp(\sigma\sqrt{\Delta t}Z) \tag{3.2}$$

Now we define a Binomial Normal Variable Z_{bin} such that $\Pr(Z_{bin} = 1) = \frac{1}{2}$ and $\Pr(Z_{bin} = -1) = \frac{1}{2}$ This has the same expected value as the Wiener Function:

$$\mathbb{E}[Z_{bin}] = (\frac{1}{2} \times 1) + (\frac{1}{2} \times -1) = 0 \tag{3.3}$$

And also the same variance as the Wiener Function:

$$\begin{aligned} \mathrm{Var}(Z_{bin}) &= \mathbb{E}[Z_{bin}^2] - (\mathbb{E}[Z_{bin}])^2 \\ \mathrm{Var}(Z_{bin}) &= (\frac{1}{2} \times (1)^2) + (\frac{1}{2} \times (-1)^2) - 0^2 \\ \mathrm{Var}(Z_{bin}) &= 1 \end{aligned} \tag{3.4}$$

We define the up factor u as the factor when $Z_{bin} = 1$ and down factor d when $Z_{bin} = -1$

$$u = \frac{S_{t+\Delta t}}{S_t}$$
$$u = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Substituting $Z_{bin} = 1$ we get the value of u to be:

$$u = e^{\sigma\sqrt{\Delta t}} \tag{3.5}$$

Doing the same for the down factor:

$$d = \frac{S_{t+\Delta t}}{S_t}$$
$$d = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Now substituting $Z_{bin} = -1$ we get the value of d to be:

$$d = e^{-\sigma\sqrt{\Delta t}} \tag{3.6}$$

When deriving the risk-neutral probability we know that the expected value of a stock at time $t + \Delta t$ is the value of that of t compounded for the time Δt or:

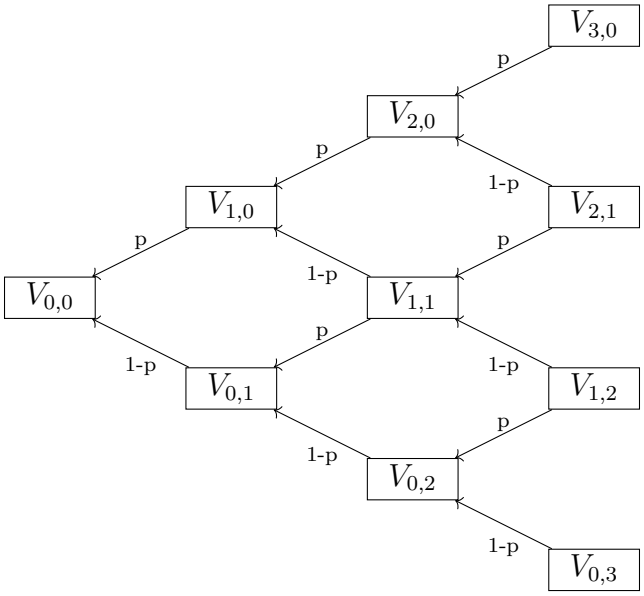
$$\mathbb{E}[S_{t+\Delta t}] = S_t e^{r\Delta t} \tag{3.7}$$

According to the binomial tree

$$\begin{aligned}
\mathbb{E}[S_{t+\Delta t}] &= S_t(up + (1-p)d) \\
S_t e^{r\Delta t} &= S_t(up + (1-p)d) \\
e^{r\Delta t} &= up + (1-p)d \\
e^{r\Delta t} &= up + d - dp \\
e^{r\Delta t} &= (u-d)p + d \\
\frac{e^{r\Delta t} - d}{u-d} &= p
\end{aligned}$$

Substituting Equations 3.5 and 3.6 for u and d respectively we get the risk neutral probability p to be:

$$p = \frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \tag{3.8}$$



each step $= \Delta t$

Figure 3: Reverse Binomial Tree to find payoffs

For the Binomial Tree, we first compute the terminal payoffs. Using the terminal payoffs we work in a reverse fashion in order to calculate the payoffs in the nodes before, with this formula:

$$V_{a,b} = e^{-r\Delta t}(pV_{a+1,b} + (1-p)V_{a,b+1}) \tag{3.9}$$

In order to create the terminal payoffs for a Binomial Tree of size n, the formula for a European Call is:

$$V_{k,n-k} = \max(u^{k-1}d^{n-k+1}S_{0,0} - K, 0) \tag{3.10}$$

where $0 \leq k \leq n$
For a put European Put option, the terminal payoffs are:

$$V_{k,n-k} = \max(K - u^{k-1}d^{n-k+1}S_{0,0}, 0) \tag{3.11}$$

3.1.2 Binomial Tree for BS Call Options

```cpp
#include <cmath>
#include <vector>
#include <type_traits>
#include "../Headers/functions.hpp"
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t,
↪  T_t>& data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);

    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};
    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    std::vector<commonType> V_current (N + 1);
    for (int j {}; j < N + 1; ++j)
    {
        V_current[j] = std::max(data.spot *
        ↪  std::pow(static_cast<double>(u) , static_cast<double>(j)) *
        ↪  std::pow(static_cast<double>(d), static_cast<double>(N-j))
        ↪  - data.strike , static_cast<commonType>(0));
    }

    for (int i {}; i < N ; ++i)
    {
        std::vector<commonType> V_new (N-i);
        for (int k {}; k < N-i; ++k)
        {
            V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
        }
        std::swap(V_current, V_new);
    }

    return V_current[0] * std::exp(-data.rate * data.maturity);
}
```

Listing 3.1: Black-Scholes-Merton Put using Monte Carlo Simulations

This function **binomialTreeBSCall** passes in the struct **OptionDataBS** and another integer parameter **N** which represents the total computes Equation 3.5 in line 13, Equation 3.6 in line 14 and Equation 3.8 in line 15.
Lines 17-20 basically is evaluating Equation 3.10 and storing it in a vector of size $N + 1$. This is then used at the start of line 23.
The mechanism here is that it first initialises a new vector that represents the column of nodes at the previous step.
Line 28 evaluates Equation 3.9 without the discount factor $e^{-r\Delta t}$. After it is done evaluating that, in line 30 it swaps the vectors, so we can use it again.
The loop variable **i** is incremented up a value, and a new vector $\mathrm{V}_{\mathrm{new}}$ is defined with a length of $N - i$. This continues untill $i = N - 1$ when the loop stop since the vector$\mathrm{V}_{\mathrm{current}}$ has a length of only 1 value. This is the final terminal payoff. Finally in line 33 this value is returned, as well as discounted with a factor of $e^{-r(T-t)}$.

3.1.3 Time Complexity of Binomial Trees

The time complexity of this algorithm mainly comes from the double nested loops from lines 23-31. The total number of iterations in the program is:

$$\begin{aligned} N_{iterations} &= N + \sum_{i=0}^{N-1}(N - i) \\ N_{iterations} &= N + N^2 - \frac{N(N-1)}{2} \\ N_{iterations} &= \frac{N^2 + 3N}{2} \end{aligned} \tag{3.12}$$

Substituting this into the time complexity expression:

$$\mathcal{O}(\frac{N^2 + 3N}{2})$$

Since we take the term with the highest order and exclude constants, the final time complexity of the Binomial Tree is:

$$\boxed{\mathcal{O}(N^2)} \tag{3.13}$$

3.1.4 Binomial Tree for BS Put Options

```cpp
#include <cmath>
#include <vector>
#include <type_traits>
#include <algorithm>
#include "../Headers/black76_structs.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t,
↪  T_t>& data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);

    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};
    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    std::vector<commonType> V_current (N + 1);
    for (int j {}; j < N + 1; ++j)
    {
        V_current[j] = std::max(data.strike - data.spot *
        ↪  std::pow(static_cast<double>(u) , static_cast<double>(j)) *
        ↪  std::pow(static_cast<double>(d), static_cast<double>(N-j))
        ↪  , static_cast<commonType>(0));
    }

    for (int i {}; i < N ; ++i)
    {
        std::vector<commonType> V_new (N-i);
        for (int k {}; k < N-i; ++k)
        {
            V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
        }
        std::swap(V_current, V_new);
    }

    return V_current[0] * std::exp(-data.rate * data.maturity);
}
```

Listing 3.2: Black-Scholes-Merton Put using Monte Carlo Simulations

This is exactly the same as the European Binomial Tree in Listing 3.1 except in line 20 it uses Equation 3.11 instead of Equation 3.11.

3.2 Binomial Trees with Black-76 Options

3.2.1 Deriving the Risk-Neutral Probability for Futures

A Binomial Tree is a numerical method that is produced by creating values until the time to maturity. The nodes at maturity are all the payoffs, and with the binomial tree we work backwards to find the price of the option at the initial time.

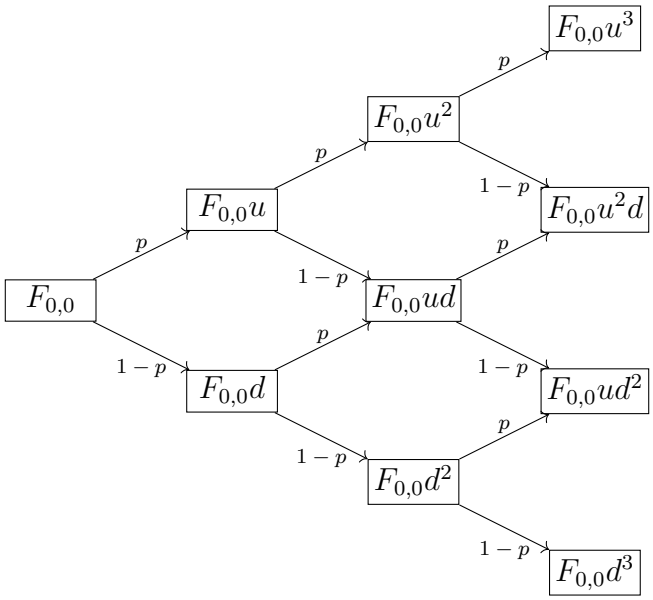


Figure 4: Binomial Tree for the future price

This is what a binomial tree for a future looks like. It can go up with probability p, and go down with probability $1-p$. The stock $F_{a,b}$ is the node that is up a nodes and down b nodes

$$F_{a,b} = u^a d^b F_{0,0} \tag{3.14}$$

In order to derive the values for u and d we are going to use the equation of a stock from Equation 2.16

$$F_T = F_t \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T-t))$$

On an interval of time Δt we have

$$\frac{F_{t+\Delta t}}{F_t} = \frac{F_t \exp((\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T-t))}{F_t}$$
$$\frac{F_{t+\Delta t}}{F_t} = \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T-t))$$

Using equation 1.2 we have:

$$W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$$
$$W_{t+\Delta t} - W_t \sim \Delta t(\mathcal{N}(0,1))$$

Letting $Z \sim \mathcal{N}(0,1)$ we have:

$$\frac{F_{t+\Delta t}}{F_t} = \exp((\sigma(\sqrt{\Delta t}Z - \frac{1}{2}\sigma^2(T-t))$$

At very small Δt , $\sqrt{\Delta t} \gg \Delta t$ which means this equation can be approximated to:

$$\frac{F_{t+\Delta t}}{F_t} \approxeq \exp(\sigma\sqrt{\Delta t}Z) \tag{3.15}$$

Using the same method for the Black-Scholes by defining a normal variable Z_{bin} we solve for u like this:

$$u = \frac{F_{t+\Delta t}}{F_t}$$
$$u = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Substituting $Z_{bin} = 1$ we get the value of u to be:

$$u = e^{\sigma\sqrt{\Delta t}} \tag{3.16}$$

And for d like this:

$$d = \frac{F_{t+\Delta t}}{F_t}$$
$$d = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Now substituting $Z_{bin} = -1$ we get the value of d to be:

$$d = e^{-\sigma\sqrt{\Delta t}} \tag{3.17}$$

When deriving the risk-neutral probability we know that the expected value of a stock at time $t + \Delta t$ is the value of that of t compounded for the time Δt or:

$$\mathbb{E}[F_{t+\Delta t}] = F_t \tag{3.18}$$

According to the binomial tree

$$\begin{aligned}
\mathbb{E}[F_{t+\Delta t}] &= F_t(up + (1-p)d) \\
F_t &= F_t(up + (1-p)d) \\
1 &= up + (1-p)d \\
1 &= up + d - dp \\
1 &= (u-d)p + d \\
\frac{e^{r\Delta t} - d}{u-d} &= p
\end{aligned}$$

Substituting Equations 3.5 and 3.6 for u and d respectively we get the risk neutral probability p to be:

$$p = \frac{1 - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \tag{3.19}$$

For the Binomial Tree, we first compute the terminal payoffs. Using the terminal payoffs we work in a reverse fashion in order to calculate the payoffs in the nodes before, with this formula:

$$V_{a,b} = e^{-r\Delta t}(pV_{a+1,b} + (1-p)V_{a,b+1}) \tag{3.20}$$

In order to create the terminal payoffs for a Binomial Tree of size n, the formula for a European Call is:

$$V_{k,n-k} = \max(u^{k-1}d^{n-k+1}F_{0,0} - K, 0) \tag{3.21}$$

where $0 \leq k \leq n$
For a put European Put option, the terminal payoffs are:

$$V_{k,n-k} = \max(K - u^{k-1}d^{n-k+1}F_{0,0}, 0) \tag{3.22}$$

3.2.2 Binomial Tree for BS Call Options

```cpp
#include <cmath>
#include <vector>
#include <type_traits>
#include <algorithm>
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t,
↪  T_t>& data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);

    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};
    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    std::vector<commonType> V_current (N + 1);
    for (int j {}; j < N + 1; ++j)
    {
        V_current[j] = std::max(data.spot *
        ↪  std::pow(static_cast<double>(u) , static_cast<double>(j)) *
        ↪  std::pow(static_cast<double>(d), static_cast<double>(N-j))
        ↪  - data.strike , static_cast<commonType>(0));
    }

    for (int i {}; i < N ; ++i)
    {
        std::vector<commonType> V_new (N-i);
        for (int k {}; k < N-i; ++k)
        {
            V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
        }
        std::swap(V_current, V_new);
    }

    return V_current[0] * std::exp(-data.rate * data.maturity);
}
```

Listing 3.3: Black-Scholes-Merton Put using Monte Carlo Simulations

Part II
American Options

1 American Binomial Tree

2 American Trinomial Tree

3 Finite Differences

4 Longstaff-Schwartz Monte Carlo

Part III
Exotic Options

References

[1] Shreve, S. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Science & Business Media.

[2] Lalley, S. P. (n.d.). *Brownian motion*. https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf

[3] Haugh, M. (2016). *The Black-Scholes model* (IEOR E4706: Foundations of Financial Engineering lecture notes). Columbia University. https://www.columbia.edu/~mh2078/FoundationsFE/BlackScholes.pdf

[4] Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, *3*(1–2), 167–179. https://doi.org/10.1016/0304-405X(76)90024-6

[5] Buchanan, J. R. (2014). *Extensions to the Black-Scholes equation: An undergraduate introduction to financial mathematics*. Millersville University. https://sites.millersville.edu/rbuchanan/book/Extensions.pdf